

Relatório do Trabalho Prático:

Nomes: Lucas Dinesh, Guilherme Hax e Henry Ribeiro

1. Descrição da Lógica de Funcionamento do Mecanismo

O mecanismo de *In-Band Network Telemetry* (INT) implementado opera inserindo metadados de telemetria diretamente no plano de dados dos pacotes à medida que eles atravessam a rede. A arquitetura foi dividida em duas frentes: o plano de dados dos switches (programado em P4) e a borda da rede/host (programada em Python utilizando Scapy).

No plano de dados (P4), o fluxo padrão de roteamento L3 (IPv4) foi mantido, mas interceptado logo após a decisão de roteamento. Quando um pacote elegível entra na rede, o primeiro switch atua como nó de *Ingress*, inserindo um cabeçalho base chamado **IntPai** (que guarda informações globais do trajeto, como a contagem de nós e o protocolo de transporte original) e o seu próprio cabeçalho de dados **IntFilho** (contendo ID do switch, portas de entrada/saída e timestamp). Os switches subsequentes (*Transit nodes*) atuam de forma iterativa, adicionando apenas os seus respectivos cabeçalhos **IntFilho** como uma pilha (*stack*), incrementando a contagem de nós no **IntPai** e atualizando o tamanho total do IPv4. O *Parser* foi projetado com estados recursivos para extrair a pilha de forma dinâmica, e o *Deparser* remonta o pacote anexando os cabeçalhos recém-inseridos antes do payload.

Na borda (Python/Scapy), o script **receive.py** atua como um *sniffer* passivo. Ele foi programado com classes customizadas que espelham a exata estrutura de bits dos cabeçalhos P4. Ao identificar o protocolo customizado no IPv4, o script fatia cirurgicamente o pacote, separando e imprimindo o caminho percorrido (Telemetria) e, em seguida, utiliza a informação preservada no **IntPai** para reconstruir o pacote de transporte original (TCP/UDP/ICMP), resgatando o payload intacto.

2. Resultados dos Experimentos Conduzidos

Nesta seção, demonstramos o funcionamento prático da rede através de diferentes cenários de uso.

A. Teste de Conectividade Básica (Bypass de Telemetria) Para garantir a transparência da rede para tráfego de controle, implementamos uma exceção para pacotes ICMP (Ping).

- *Experimento:* Execução do comando **pingall** no Mininet.
- *Resultado:* 100% de sucesso na entrega dos pacotes. O script **receive.py** ignora essas mensagens, demonstrando que a telemetria não interfere no tráfego L3 padrão.

```
p4@p4dev: ~/Downloads/TP-protocolos/TP-protocolos/skeleton/TP-skel
its CLI from your host operating system using this command:
  simple_switch_CLI --thrift-port <switch thrift port>

To view a switch log, run this command from your host OS:
  tail -f /home/p4/Downloads/TP-protocolos/TP-protocolos/skeleton/TP-skel/logs/<switchname>.log

To view the switch output pcap, check the pcap files in /home/p4/Downloads/TP-protocolos/TP-protocolos/skeleton/TP-skel/pcaps:
  for example run: sudo tcpdump -xxx -r s1-eth1.pcap

To view the P4Runtime requests sent to the switch, check the
corresponding txt file in /home/p4/Downloads/TP-protocolos/TP-protocolos/skeleton/TP-skel/logs:
  for example run: cat /home/p4/Downloads/TP-protocolos/TP-protocolos/skeleton/TP-skel/logs/s1-p4runtime-requests.txt

mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
mininet> 
```

B. Rastreamento de Caminho e Extração de Payload (INT Padrão)

- *Experimento*: Envio de uma string de texto via protocolo UDP/TCP utilizando o [send.py](#):
- do host 1 para o host 2 e host 3
- do host 2 para o host 1 e para o host 3
- do host 3 para o host 1 e host 2.
- *Resultado*: O pacote chega aos hosts destino encapsulado com o INT. O **send.py** exibe a lista reversa dos switches visitados (do Ingress ao Egress) contendo as portas e o *timestamp*. Logo abaixo, o script isola os bytes da telemetria e reconstrói o payload de forma estritamente separada, exibindo a mensagem original enviada.


```
"Node: h1"
##### Ethernet #####
dst = ff:ff:ff:ff:ff:ff
src = 08:00:00:00:00:01:11
type = IPv4
##### IP #####
version = 4
ihl = 5
tos = 0x0
len = 32
id = 1
flags = 0
frag = 0
ttl = 64
proto = tcp
chksum = 0x63e1
src = 10.0.1.1
dst = 10.0.2.2
options \
##### TCP #####
sport = 63467
dport = 3234
seq = 0
ack = 0
dataofs = 5
reserved = 0
flags = S
window = 8192
chksum = 0x44ff
urgstr = 0
options = []
##### Raw #####
load = 'hello nodo 2'

root@h4dev:/home/p4/Downloads/IP-protocolos/skeleton/IP-skel# python3 send.py 10.0.3.3 "hello nodo 3"
##### Ethernet #####
dst = ff:ff:ff:ff:ff:ff
src = 08:00:00:00:00:01:11
type = IPv4
##### IP #####
version = 4
ihl = 5
tos = 0x0
len = 32
id = 1
flags = 0
frag = 0
ttl = 64
proto = tcp
chksum = 0x62e0
src = 10.0.1.1
dst = 10.0.3.3
options \
##### TCP #####
sport = 63361
dport = 3234
seq = 0
ack = 0
dataofs = 5
reserved = 0
flags = S
window = 8192
chksum = 0x447b
urgstr = 0
options = []
##### Raw #####
load = 'hello nodo 3'

"Node: h2"
root@h4dev:/home/p4/Downloads/IP-protocolos/skeleton/IP-skel# python3 receive.py
Iniciando sniffer na interface eth0 aguardando tráfego INT...

[+] PACOTE COM TELEMETRIA (INT) RECEBIDO!
=====
> Protocolo L4 Original: 6
> Quantidade de Saltos: 2

[OK] Status do MTU: Saudável, Todos os dados foram coletados.

--- Caminho Percorrido ---
Salto 1: Switch ID = 1 | Porta In: 1 -> Porta Out: 2 | Timestamp: 232033902
Salto 2: Switch ID = 2 | Porta In: 2 -> Porta Out: 1 | Timestamp: 231533110

--- Payload Original (TCP) ---
Mensagem: hello nodo 2

"Node: h3"
root@h4dev:/home/p4/Downloads/IP-protocolos/skeleton/IP-skel# python3 receive.py
Iniciando sniffer na interface eth0 aguardando tráfego INT...

[+] PACOTE COM TELEMETRIA (INT) RECEBIDO!
=====
> Protocolo L4 Original: 6
> Quantidade de Saltos: 2

[OK] Status do MTU: Saudável, Todos os dados foram coletados.

--- Caminho Percorrido ---
Salto 1: Switch ID = 1 | Porta In: 1 -> Porta Out: 3 | Timestamp: 252946372
Salto 2: Switch ID = 3 | Porta In: 2 -> Porta Out: 1 | Timestamp: 252023938

--- Payload Original (TCP) ---
Mensagem: hello nodo 3
```

Mensagem do host 1 para o host 1 e host 3

C. Simulação de Prevenção de Fragmentação (Requisito Bônus - MTU)

Experimento: Envio de um pacote estressante utilizando o comando **python3 send.py 10.0.2.2 \$(python3 -c "print('A' * 1410)")**.

Resultado: O pacote entra no limite aceitável no primeiro switch e recebe telemetria. Ao chegar no segundo switch, o hardware detecta que a inserção do **IntFilho** ultrapassaria o MTU de 1500 bytes. O switch aborta a inserção e acende a **flag** de estouro (campo **estouro_mtu = 1**).

- *Problema:* Ao sobrescrever o **ipv4.protocol** com 253, perdíamos a referência do próximo protocolo esperado (TCP/UDP), o que impedia a aplicação final de reconstruir o pacote.
 - *Solução:* Resolvemos o problema adicionando um campo **proto_original** de 8 bits dentro da própria estrutura do **IntPai**. O primeiro switch salva o protocolo nativo ali antes de alterar para 253. O Python utiliza esse campo para remontar a camada de transporte dinamicamente.
3. **Iteração sem Laços de Repetição no P4:**
- *Problema:* A linguagem P4 não possui comandos como *for* ou *while* para extrair uma quantidade dinâmica de cabeçalhos inseridos por múltiplos switches.
 - *Solução:* Utilizamos a recursividade no bloco *Parser*. O estado de *parse* do filho aponta para o próximo elemento (**hdr.int_filhos.next**) e utiliza a função **lastIndex** comparada à variável global **Quantidade_Filhos** para decidir se continua iterando ou se aceita o pacote.
4. **Ordem de Execução e Rotas (Porta de Saída):**
- *Problema:* Inicialmente, o campo **Porta_Saida** registrava valores vazios ou incorretos.
 - *Solução:* Compreendemos que a porta de saída (**standard_metadata.egress_spec**) só se torna uma variável conhecida após a tabela de roteamento decidir o caminho. Ajustamos o código para que o bloco INT fosse executado estritamente após a chamada **ipv4_lpm.apply()**.
5. **Identificação dos Switches no Plano de Controle:**
- *Problema:* Os switches precisavam gravar seus respectivos IDs no pacote, mas o P4 não possui uma variável nativa para "nome do nó".
 - *Solução:* Criamos um registro na estrutura *metadata* (**meta.switch_id**) que é preenchido através de uma tabela controlada via *Control Plane* (arquivos **sX-runtime.json**), permitindo que a atribuição de IDs fosse dinâmica.
6. **Falha do Pingall (Incompatibilidade com o Kernel do Linux):**
- *Problema:* O comando **pingall** resultava em 100% de perda. O switch alterava o pacote ICMP para o protocolo 253. Ao chegar no host Linux, o Kernel desconhecia a estrutura, descartava o pacote e gerava mensagens de erro do tipo *ICMP protocol-unreachable*.
 - *Solução:* Implementamos uma exceção condicional (**if (hdr.ipv4.protocol != 1)**). Pacotes de ping não recebem telemetria, mantendo o comportamento original da rede e contornando a falha sem afetar o monitoramento de aplicações reais (TCP/UDP).
7. **O Problema de Parsing Guloso no Scapy (Python):**
- *Problema:* No **receive.py**, a biblioteca fundia o *payload* (mensagem "hello") com valores numéricos absurdos dentro dos campos do **IntFilho**.
 - *Solução:* Utilizamos o recurso **PacketListField** no Scapy atrelado ao **count_from**. Isso forçou o Python a ler exatamente a quantidade de filhos determinada pelos metadados, isolando o restante do pacote adequadamente na camada **Raw**.
8. **Cálculo de Bytes e Type Casting (Limite de MTU):**
- *Problema:* O compilador P4 recusava a soma matemática para controle de MTU devido a conflitos entre variáveis **bit<32>** (tamanho do pacote) e

bit<16> (tamanho do IPv4). Além disso, não podíamos basear o cálculo de MTU no campo **totalLen** do IP, pois ele omite o peso do cabeçalho Ethernet.

- *Solução:* Utilizamos a variável de hardware **standard_metadata.packet_length** para precisão física. Após a checagem, forçamos o alinhamento de memória usando Type Casting explícito **((bit<16>)tamanho_adicional)** para permitir a atualização do IPv4 de forma segura.

4. Breve Descrição dos Passos para Implementação de Cada Requisito Funcional

- **Requisito: Inserção do Cabeçalho PAI e FILHO nos Switches**
 - *Passo 1:* Definição das estruturas **int_pai_t** (12 bytes) e **int_filho_t** (16 bytes) no topo do arquivo P4. Instanciação de um array **int_filho_t[10]** na struct **headers**.
 - *Passo 2:* Construção do **MyParser** para identificar **protocol == 253** e transitar para a extração do **IntPai** e, sucessivamente, dos **IntFilhos**.
 - *Passo 3:* No **MyIngress**, implementação da lógica que distingue o 1º nó (que insere ambos, ativando **setValid()**) dos nós de trânsito (que inserem apenas filhos utilizando **push_front(1)** e atualizando a variável global de contagem de saltos).
 - *Passo 4:* Inclusão dos headers no **MyDeparser** para serialização de saída (**packet.emit**).
- **Requisito: Extração e Separação de Dados no Host Destino (receive.py)**
 - *Passo 1:* Modelagem das classes **IntPai** e **IntFilho** herdando da classe **Packet** do Scapy, obedecendo ao tamanho exato de bits definidos no P4. Amarração da camada IP ao **IntPai** via **bind_layers**.
 - *Passo 2:* Criação do método **handle_pkt** para filtrar ruídos da rede, invertendo a lista de switches para exibição cronológica (Origem → Destino).
 - *Passo 3:* Utilização da estrutura condicional para extrair a carga útil remanescente (**pkt[Raw]**), convertendo-a novamente em um objeto TCP/UDP utilizando o **proto_original** salvo pelo switch de Ingress, garantindo a visualização separada da telemetria e da mensagem.
- **Requisito Bônus: Prevenção de Estouro de MTU**
 - *Passo 1:* Adição de uma flag de 1 bit (**estouro_mtu**) no cabeçalho **IntPai** (P4 e Python), subtraindo 1 bit do campo de *padding* para manter o alinhamento de memória nativo.
 - *Passo 2:* Implementação de uma variável preditiva (**tamanho_adicional**) no **MyIngress** do P4. Antes de inserir qualquer dado, o código soma essa variável ao **standard_metadata.packet_length**.
 - *Passo 3:* Uso de estrutura lógica: se a soma for menor ou igual a 1500, a telemetria é acoplada. Se for maior, o switch ignora a adição e apenas altera a flag **estouro_mtu = 1**.
 - *Passo 4:* Configuração do **receive.py** para inspecionar essa flag e emitir um aviso visual ao usuário sobre o abandono de telemetria devido ao risco de fragmentação.